

TP4 - Indexation de Texte par ABR

Timothée POUPINEAU & Loudiern THARON

17 décembre 2025

1 Structures et Fonctions Supplémentaires

1.1 Structures Supplémentaires

En plus des structures demandées (**T_Position**, **T_Noeud**, **T_Index**), nous avons implémenté :

- **T_MotOrdre** (B.6) : Liste chaînée stockant un mot et son ordre dans la phrase. Permet de reconstruire une phrase en ordre croissant lors de l’affichage des occurrences.
- **T_Phrase** (B.7) : Tableau dynamique contenant les mots d’une phrase avec leurs ordres. Facilite la reconstruction complète du texte.

Justification : Ces structures évitent des parcours répétés de l’ABR et permettent de trier les mots efficacement pour reconstituer les phrases dans le bon ordre.

1.2 Fonctions Auxiliaires

- **initialiserIndex()** : Initialise un index vide
- **convertirMinuscules()** : Ignore la casse (voiture = Voiture = VOITURE)
- **libererPositions()**, **libererNoeud()**, **libererIndex()** : Libération mémoire
- **afficherNoeudInfixe()** : Parcours infixe pour affichage alphabétique
- **ajouterMotOrdre()** : Insertion triée dans une liste de mots ordonnés
- **collecterMotsPhrase()** : Collecte les mots d’une phrase donnée
- **afficherPhrase()**, **libererMotsOrdres()** : Affichage et libération pour B.6
- **ajouterMotPhrase()**, **collecterPhrasesRecursif()** : Collecte pour B.7

Justification : Modularité du code, réutilisation des fonctions et gestion rigoureuse de la mémoire dynamique.

2 Complexité des Fonctions

2.1 Fonctions Demandées (B.1 à B.7)

Fonction	Complexité	Remarque
ajouterPosition()	$O(p)$	p = nb positions du mot
ajouterOccurence()	$O(h + p)$	h = hauteur ABR
indexerFichier()	$O(m \times h)$	m = nb mots fichier
afficherIndex()	$O(n \times p)$	n = mots distincts
rechercherMot()	$O(h)$	Recherche binaire
afficherOccurrencesMot()	$O(k \times n)$	k = occurrences du mot
construireTexte()	$O(n \times p + s^2)$	s = nb phrases

Notation : h = hauteur de l’arbre. Cas moyen : $h = O(\log n)$. Pire cas : $h = O(n)$.

2.2 Fonctions Auxiliaires

Fonction	Complexité	Remarque
<code>initialiserIndex()</code>	$O(1)$	Initialisation constante
<code>convertirMinuscules()</code>	$O(l)$	l = longueur du mot
<code>afficherNoeudInfixe()</code>	$O(n \times p)$	Parcours infixe récursif
<code>libererPositions()</code>	$O(p)$	Parcours linéaire de la liste
<code>libererNoeud()</code>	$O(n \times p)$	Libération récursive
<code>libererIndex()</code>	$O(n \times p)$	Appelle <code>libererNoeud()</code>
<code>ajouterMotOrdre()</code>	$O(m)$	m = mots par phrase
<code>collecterMotsPhrase()</code>	$O(n \times p)$	Parcours complet ABR
<code>afficherPhrase()</code>	$O(m)$	Parcours linéaire
<code>libererMotsOrdres()</code>	$O(m)$	Libération linéaire
<code>ajouterMotPhrase()</code>	$O(1)$ amorti	Doublement capacité
<code>collecterPhrasesRekursif()</code>	$O(n \times p \times s)$	Parcours ABR + recherche phrase

3 Choix d'Implémentation

- **Casse** : Conversion systématique en minuscules via `tolower()`.
- **Ordre** : `strcmp()` pour l'ordre lexicographique dans l'ABR.
- **Positions triées** : Insertion en $O(p)$ mais évite des tris ultérieurs.
- **Mémoire** : Tableaux dynamiques avec doublement de capacité. Libération complète à la fin.
- **B.6 optimisé** : Pour chaque occurrence, parcours unique de l'ABR pour collecter les mots de la phrase concernée, insertion triée par ordre.
- **B.7** : Collecte globale des phrases puis tri par numéro et par ordre des mots.